PX915 Software Development Plan

Tristan McCarthy, Jasper Allen, Stephan Gambart, Lucas Belz-Koeling.

March 2026

1 Specification

1.1 Basic features

- The project will produce a traffic flow simulation software package to model vehicle dynamics in road networks.
- The core functionality will include a working implementation of a 1D cellular automaton traffic model [1], representing a single lane of unidirectional traffic.
- The model will simulate vehicle movement on a discretised road consisting of cells that may be occupied by a vehicle or empty, with motion governed by defined update rules.
- The software will allow users to run simulations with configurable parameters, such as road length, vehicle entry rate, and exit rate.
- The program will generate traffic metrics of interest, including:
  - vehicle density
  - traffic flow
  - travel time through a road segment;
  - fundamental diagrams relating flow and density.
- The software package will be completed and ready for use by the 28th of May 2026.

1.2 Specific features

- The software will support simulations of road networks that contain bifurcations (one road splits into two) and merging roads (two roads split into two), allowing vehicles to transition between connected road segments according to defined routing rules.
- The base 1D cellular automaton model will be extended to a 2D grid representation of roads. This will enable modelling of multiple parallel traffic lanes and allow vehicles to change lanes under defined conditions.
- The multi-lane model will also enable the simulation of lane changes resulting from decision-based traffic splitting at junctions.

1.3 Extension features

These features will only be implemented once the core cellular automaton traffic model is complete and validated.

- Implement a continuum traffic flow model based on a partial differential equation describing the evolution of vehicle density as a function of space and time.

  - The PDE model will represent traffic as a continuous population flow, allowing comparison with the discrete cellular automaton model used in the core implementation.
  - Simulation outputs from the continuum model will be compared against the discrete cellular automaton simulations to evaluate consistency between the two modelling approaches.

- Include the option to simulate a diverse set of vehicles, such as lorries, bicycles, or pedestrians.
- Implement a Nagel-Schreckenberg type model [2], where each vehicle is modelled with an individual, variable velocity. This will allow the simulation of variable acceleration/braking behaviour, as well as overtaking, introducing more realistic traffic interactions.

2 Design

2.1 Model Specifications

2.1.1 TASEP

The Totally Asymmetric Simple Exclusion Process (TASEP) is used as the core microscopic traffic model in this project [1]. TASEP is a stochastic cellular automaton defined on a one–dimensional lattice representing a single lane of traffic.

The road is discretised into L cells, where each cell i can either be occupied by a vehicle or empty. The state of the system at time t is represented by

$$s_i(t) \in \{0, 1\}, \qquad i = 1, 2, \ldots, L, \tag{1}$$

where $s_i = 1$ indicates that a vehicle occupies cell i, and $s_i = 0$ indicates that the cell is empty.

Vehicle motion follows the exclusion principle: a vehicle may only move forward if the cell directly in front of it is empty. The system evolves in discrete time according to stochastic update rules with open boundaries.

Vehicles may enter the road at the upstream boundary and exit at the downstream boundary. The entry and exit processes are controlled by the parameters α and β, which represent the probabilities of vehicle injection and removal respectively.

In the bulk of the lattice, vehicle motion occurs with an occupancy dependent hopping rate. A vehicle at site i attempts to move to site $i + 1$ provided that the destination cell is empty. The transition rate can be written as

$$i \to i + 1 \quad \text{with rate } r(s_i, s_{i+1}), \tag{2}$$

where the hopping rate r may depend on the occupancies of the local cells. This allows the model to capture the effect of local congestion, since the probability of movement depends on the surrounding vehicle configuration.

The model therefore evolves according to the following rules:

- **Entry:** A vehicle enters the first cell with probability α if $s_1 = 0$.
- **Bulk motion:** For $i = 1, \ldots, L-1$, a vehicle at site i moves to site $i+1$ with rate $r(s_i, s_{i+1})$, provided that the next cell is empty.
- **Exit:** A vehicle at the final cell exits the system with probability β.

These rules define an open-boundary TASEP model in which vehicles continuously enter and leave the system. The parameters α and β determine the traffic regime by controlling the injection and removal rates at the boundaries.

The simulation records several macroscopic quantities of interest, including:

- **Vehicle density**

$$\rho = \frac{1}{L} \sum_{i=1}^{L} s_i \tag{3}$$

- **Traffic flow (current)**, measured as the number of vehicles exiting the road per unit time
- **Fundamental diagrams**, which describe the relationship between traffic flow and density

In the initial implementation used for the toy model, the hopping probability is taken to be $r = 1$, meaning that vehicles always move forward whenever the next cell is empty. This provides a simple baseline model which can later be extended to more complex traffic models such as the Nagel–Schreckenberg model or continuum PDE descriptions of traffic flow.

Figure 1: Illustration of the TASEP model. Vehicles (filled circles) move to the right with an occupancy-dependent rate if the next cell is empty. Vehicles enter with rate α and exit with rate β.

2.1.2 Nagel-Schreckenberg Model

The Nagel–Schreckenberg model is an extension of the TASEP model, implementing individual, variable velocities to each vehicle, producing more realistic driving behaviours [2].

At each discrete time step, all vehicles are updated synchronously according to the model rules. Each vehicle can perform a number of actions, depending their current velocity, their max velocity (speed limit), and the gap between it and the vehicle in front. The max velocities are assigned based on some random distribution at the model's initiation.

For each vehicle i, the following steps are performed:

1. **Acceleration**
$$v_i \leftarrow \min(v_i + 1, v_{\max}) \tag{4}$$
2. **Deceleration (collision avoidance)**
$$v_i \leftarrow \min(v_i, g_i) \tag{5}$$
where the gap is defined as
$$g_i = x_{i+1} - x_i - 1 \tag{6}$$
3. **Randomisation**
With probability p,
$$v_i \leftarrow \max(v_i - 1, 0) \tag{7}$$
4. **Movement**
$$x_i \leftarrow (x_i + v_i) \bmod L \tag{8}$$

All updated positions and velocities are written to a new lattice state, which replaces the old state at the end of the time step.

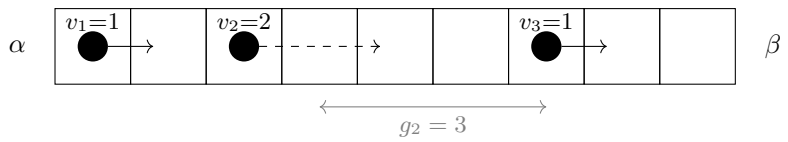


Figure 2: Illustration of how the Nagel-Schreckenberg model may be set up in a given timestep

Efficient gap computation is critical for performance. Two approaches are considered:

- **Naïve approach:** scanning forward from each vehicle until the next occupied cell is found.

- **Optimised approach:** maintaining a sorted list of vehicle positions and computing gaps using neighbour indexing.

The optimised approach reduces computational complexity per vehicle and is preferred for large systems, reducing the amount of necessary message passing between individual vehicles.

The design allows for future extensions, including:

- Multi-lane traffic models
- Heterogeneous driver behaviour (variable p, $v_{\max}$)
- Simulation of emergent traffic behaviour, such as spontaneous traffic jams

2.1.3 PDE

A typical model, like the Lighthill-Whitham-Richards model [3] , has the form:

$$\boxed{\frac{\partial \rho}{\partial t} + \frac{\partial}{\partial x}(\rho v(\rho)) = 0,} \tag{9}$$

where $\rho(x,t)$ represents the traffic density and $v(\rho)$ is the velocity-density relationship. Some details about how the PDE will be solved are:

- **Fundamental diagram** – the relationship between density and velocity (or flux) will be specified through a fundamental diagram. For example, a simple choice is a decreasing velocity function $v(\rho)$, leading to a flux function $q(\rho) = \rho v(\rho)$ with a maximum flow at intermediate densities.
- **Numerical solution** – the PDE will be solved numerically using a finite volume or finite difference method. This allows the conservation law structure to be preserved and enables accurate capture of discontinuities such as shock waves.
- **Initial and boundary conditions** – appropriate initial density profiles and boundary conditions will be defined to represent realistic traffic scenarios, such as open boundaries or fixed inflow/outflow rates.
- **Model comparison** – results from the PDE model will be compared with the cellular automaton model using consistent initial conditions to assess similarities and differences in traffic behaviour.
- **Model limitations** – the macroscopic model assumes traffic behaves as a continuous fluid and does not track individual vehicles, which may limit its ability to capture microscopic effects such as individual driver behaviour or lane-changing dynamics.
- **Validation** – the model will be validated by checking for known behaviours such as the formation and propagation of shock waves and comparison with expected traffic flow patterns.

The PDE will be designed to allow additional modelling of traffic behaviours such as:

- **overtaking** – although individual vehicles are not tracked in a macroscopic model, overtaking effects are incorporated through the velocity-density function $v(\rho)$, where lower densities correspond to higher average vehicle speeds.
- **road bifurcations** – junction conditions can be introduced where the road splits, allowing the traffic flux to be divided between outgoing roads according to a specified proportion of vehicles taking each route.
- **merging traffic streams** – when two incoming roads merge, the combined incoming flux must satisfy the capacity of the downstream road, which may lead to congestion and increased density upstream.
- **other macroscopic traffic effects** – the PDE framework naturally captures large-scale behaviours such as traffic shock waves and stop-and-go patterns that propagate through the traffic density.

2.2 Overall Architecture

The software will be structured as a modular simulation framework consisting of a core simulation engine and supporting modules for input, analysis, and visualisation.

- **Simulation module**: Implements the traffic model dynamics.
- **Road network representation**: Stores the structure of the road network and vehicle positions.
- **Analysis module**: Computes traffic metrics such as density and flow.
- **Input / output module**: Handles configuration files and data storage.
- **Visualisation module**: Produces plots such as fundamental diagrams and space-time diagrams.

This modular design allows different traffic models (e..g. cellular automaton or PDE models) to be implemented within the same framework.

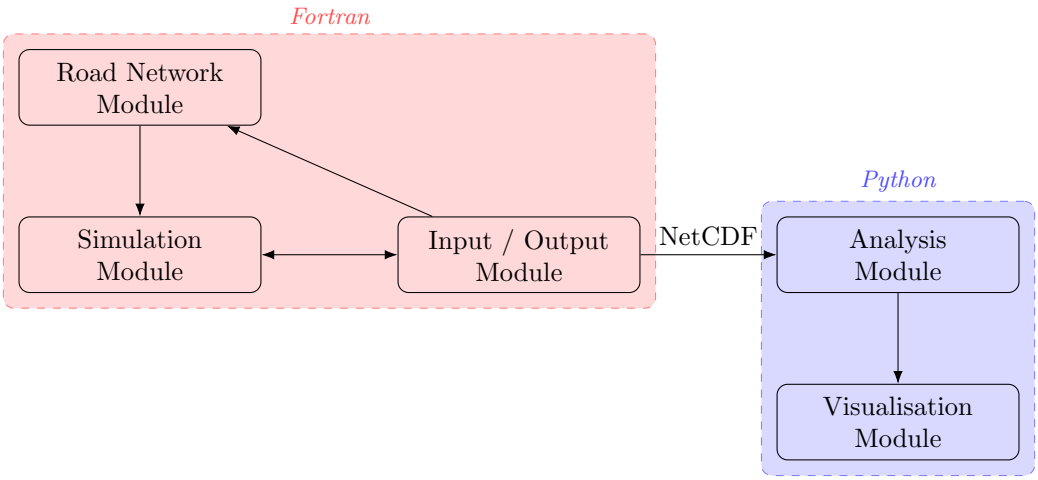


Figure 3: High-level software architecture of the traffic flow simulation package.

2.3 Module Interfaces

The interaction between modules follows the architecture shown in Figure 2:

- **Input/Output Module → Road Network Module:** Provides configuration data defining the road topology. The Python interface writes road parameters into the NetCDF header before execution, from which the Fortran I/O module reads them into the road network representation.
- **Input/Output Module → Simulation Module:** Supplies simulation parameters such as lattice size, α, β, and number of timesteps. These are specified in Python (either as fixed values or as a sweep range) and written to the NetCDF header, from which Fortran reads them into the simulation. All parameters are stored in the NetCDF header and are available for exact restarts.
- **Road Network Module → Simulation Module:** Provides the structural representation of the road network used during simulation as a `RoadNetwork` derived type.
- **Simulation Module → Input/Output Module:** At every `output_interval` time step the simulation writes the full system state, current time step, and exit flow into the NetCDF file. Checkpoint fields are additionally written at every `checkpoint_interval`.

- **Input/Output Module → Analysis Module:** Outputs simulation data via the NetCDF file, which the Python analysis module reads directly. The file contains the full state history, flow time series, and the original parameter header.
- **Analysis Module → Visualisation Module:** Provides processed data for plotting fundamental diagrams and visualisation of the traffic on the road. Both modules operate exclusively on the NetCDF file, with the analysis module passing computed arrays directly to the visualisation module without further file I/O.

2.4 Core Modules

The codebase will be organised into the following logical modules:

- **Simulation Module**: Responsible for performing the time evolution of the traffic system.
  - implement the TASEP cellular automaton update rules
  - may implement the PDE based continuum traffic flow model
  - update vehicle positions at each time step
  - handle vehicle entry and exit boundary conditions
- **Road Network Module**: Represents the spatial structure of the traffic system.
  - store road geometry and connectivity
  - represent road segments as arrays of cells
  - manage connections between roads (bifurcations and merges)
- **Analysis Module**: Computes quantities of interest from simulation outputs.
  - calculate vehicle density
  - compute traffic flow
  - generate fundamental diagrams
  - measure travel time statistics
- **Input/Output Module**: Handles user inputs and simulation results.
  - accept simulation parameters as input from either command line or Python user interface
  - store simulation outputs (e.g. NetCDF or CSV)
  - allow results to be reused for analysis
- **Visualisation Module**: Provides tools for visualising simulation results.
  - plot density vs time
  - produce space–time diagrams
  - generate flow–density plots

2.5 Programming Languages and Tools

The software will use a hybrid Python–Fortran architecture.

- **Fortran** will be used for the core simulation routines, including the cellular automaton update rules and time evolution of the traffic model. This code will be compatible with the Fortran 2008 standard.
- **Python** will provide the high-level interface for running simulations, analysing results, and producing visualisations. This code will be compatible with Python 3 and a `requirements.txt` file, specifying all library versions, will be provided.

- Scientific Python libraries such as `NumPy` and `Matplotlib` will be used for numerical analysis and plotting.
- Simulation data from Fortran will be stored in a Python-interpretable format using `NetCDF`.
- Version control and collaboration will be managed using **GitHub**.

2.6 Parallelisation

The simulation itself will likely be serial, but parameter sweeps (e.g. varying α and β to construct fundamental diagrams) are embarrassingly parallel. These will be parallelised using **OpenMP** over a parameter loop, with each thread running an independent simulation instance. No shared mutable state exists between runs, so no synchronisation is required beyond a reduction to collect results.

2.7 Checkpoint and Restart

The simulation will support periodic check pointing every C time steps, where C is a user configurable parameter. Checkpoints are written by the I/O module to a NetCDF file and a simulation can be resumed by passing the checkpoint file path at the command line.

To guarantee an exact restart the following data must be stored, per model:

- **TASEP:** `lattice(1:L)`, `t_step`, `rng_state`, and parameters α, β, L, r, C.
- **Nagel–Schreckenberg:** `positions(1:N)`, `velocities(1:N)`, `v_max(1:N)`, `t_step`, `rng_state`, and parameters L, p, α, β, C. The per-vehicle `v_max` array must be stored explicitly as it is sampled from a distribution at initialisation and is not re-derivable.
- **PDE:** `density(1:M)`, `t_current`, and parameters Δx, Δt, domain length, boundary values, C.

Checkpoint files use the same NetCDF schema as standard output files, with an additional `/restart` group containing the fields above.

3 Work plan

This section outlines the project work plan, covering team structure, task assignments, testing strategy, and collaboration tools.

3.1 Team Structure

The project will be divided into two parallel work streams: a discrete modelling team and a continuous modelling team.

- **Discrete model team** – two members will focus on implementing and validating the cellular automaton traffic model. This includes developing the update rules, handling boundary conditions, and ensuring the model reproduces expected traffic behaviours such as congestion and flow transitions.
  - Both Tristan and Jasper begin the 1D Toy Model in Week 0, as it forms the foundation on which all other components depend
  - Lucas reviews the cellular automaton literature in Week 0 in preparation for the CA implementation
- **Continuous model team** – two members will develop the PDE-based traffic flow model, including implementation of the governing conservation law, numerical solution methods, and incorporation of macroscopic traffic effects such as shock formation (sharp transition between free-flowing and congested traffic).
  - Stephan reads into PDE-based traffic models in Week 0 ahead of the continuum model work.

  - Jasper and Stephan develop the PDE implementation in parallel with the CA work; if time allows, Jasper will attempt UQ and Parameter Sensitivity Analysis, while Stephan carries out a comparison between models once both are functional.

- **Parallel development** – both teams will work concurrently during the early stages of the project to maximise efficiency. Each team will initially operate independently while adhering to agreed modelling assumptions and parameter definitions to ensure compatibility.

- **Interface and comparison** – a common framework will be established for comparing the two models, including consistent initial conditions, boundary conditions, and performance metrics (e.g. flow rate, density evolution).

  - Tristan leads Bifurcations and Merges, which depends on the 1D model being in place since junction handling requires a working single-lane simulation.
  - The Full Road Network (Tristan & Lucas) is the most dependent task in the project, requiring both junction handling and multi-lane behaviour to be sufficiently complete before assembly into a unified network.
  - Lucas builds on the 1D model to implement the Nagel-Schreckenberg extension, covering Acceleration & Deceleration followed by Multi-lane & Overtaking.

- **Collaboration and knowledge sharing** – regular meetings will be held to share progress, discuss challenges, and ensure that knowledge is distributed across the group, reducing the risk of knowledge gaps. When a team member has no active tasks assigned, they will monitor overall progress and provide support where needed.

- **Integration phase** – in the later stages of the project, the two work streams will be brought together to compare results, validate model behaviour, and produce unified visualisations and analysis. The Presentation draws on all completed work and is a joint responsibility across the whole team.

- **Contingency planning** – if one approach proves significantly more challenging or less successful, people can be reallocated between teams to ensure that at least one model is fully completed and validated. If time pressure arises, the multi-lane work takes priority over overtaking.

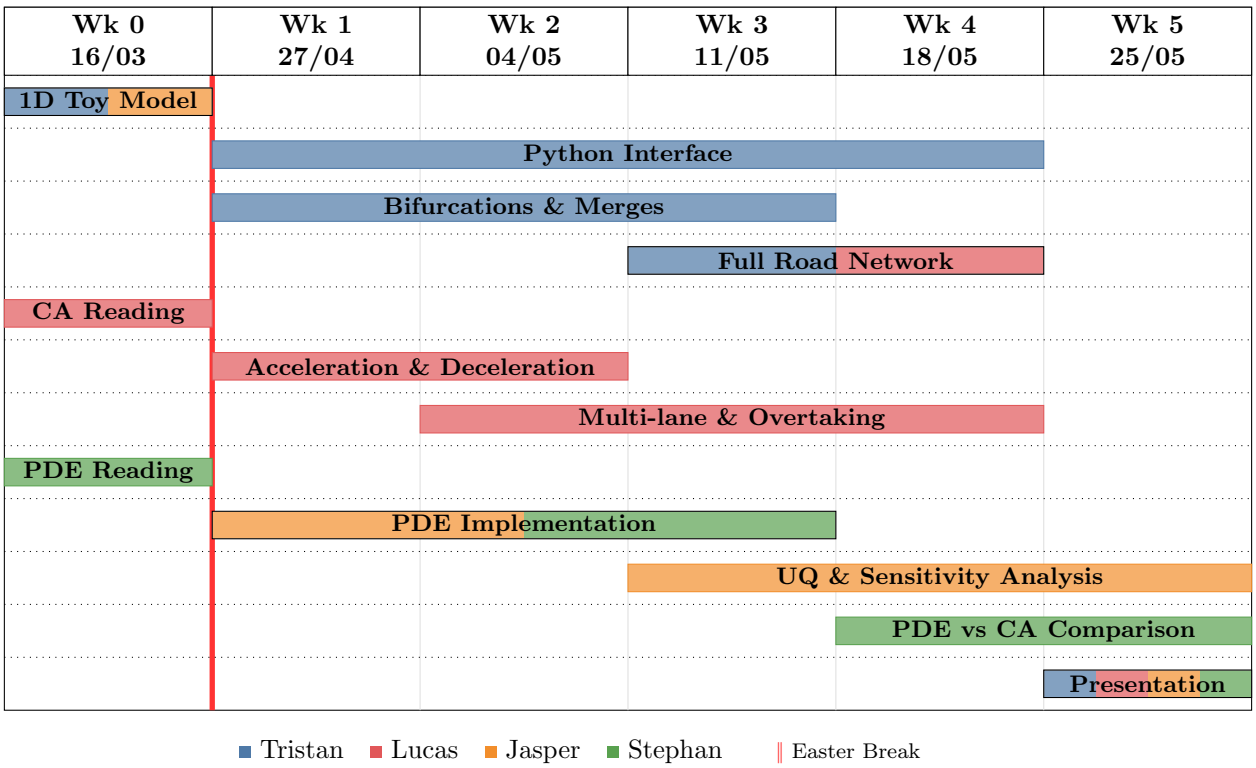
- **Milestones** – the four key milestones are: (1) a validated 1D toy model, (2) a functional PDE solver, (3) a working multi-lane road network, and (4) a completed model comparison with unified visualisations. All four must be reached before the final presentation.

- **Testing strategy** – both models will be tested independently using simple benchmark scenarios (e.g. known analytical solutions for the PDE, expected results for the CA) before integration. The shared test suite in `tests/` will be run before any merge to `main`.

- **Documentation** – code will be documented using docstrings throughout development and compiled into a readable reference using **ReadTheDocs**. The `notebooks/` directory will provide worked examples of key analyses to aid reproducibility.

3.2 Timeline

The following Gantt chart summarises the task assignments and timeline across the project period.



The Gantt chart outlines the project timeline across six weeks, with tasks sequenced to respect codebase dependencies and assigned to team members based on their areas of responsibility. Week 0 is separated from the remaining weeks by the Easter break, during which no work is expected. The software submission deadline is 28th May; the presentation is scheduled for the following Friday the 5th of June and draws on all completed work.

3.3 Collaboration Tools

The team will use **GitHub** for version control and code sharing. Each team member will work on their own branch and submit pull requests to merge changes into `main`.

The repository is structured to mirror the module design described in Section 2:

- `src/fortran/` – the core Fortran simulation routines (TASEP, Nagel-Schreckenberg, utilities)
- `src/python/` – the Python interface, with one file per module: `io.py`, `road_network.py`, `analysis.py`, and `visualisation.py`
- `scripts/` – entry-point scripts for running each simulation type
- `tests/` – a shared test suite to catch regressions before merging
- `data/` – designated folders for simulation inputs and outputs
- `notebooks/` – Jupyter notebooks for analysis and visualisation examples

Day-to-day communication will be handled via Slack, where the team will share updates and plan weekly in-person meetings.

4 Risk and Management

Risk	Description	Mitigation
Version Control Conflicts	Multiple team members editing the same files can cause merge conflicts where changes overwrite each other or break the code.	Use version control systems and coordinate commits.
Integration Problems	Different parts of the code may work individually but fail when combined due to inconsistent assumptions, incompatible interfaces, or missing dependencies.	Regular integration and testing of combined components.
Uneven Work Distribution	Some members may end up doing significantly more work than others, which can cause delays or frustration within the team.	Clear task assignments and weekly progress checks.
Lack of Communication	If team members do not regularly communicate updates or issues, work may be duplicated or dependencies missed.	Weekly progress meetings and regular communication.
Inconsistent Coding Styles	Different coding styles or naming conventions can make the code harder to read, maintain, and debug.	Establish shared coding guidelines and perform code reviews.
Knowledge Gaps	If only one person understands a critical part of the code, progress may stall if they are unavailable.	Share documentation and ensure knowledge is distributed across the team.
Bugs Introduced by Changes	New features or edits can unintentionally break existing functionality.	Regular testing and code reviews before merging changes.
Dependency or Environment Issues	Different software environments, library versions, or operating systems may cause code to run differently across machines.	Use shared environment configurations and document dependencies.
Time Management Risks	Underestimating the complexity of tasks can result in missed deadlines.	Plan milestones and monitor progress regularly.
Lack of Testing	Without proper testing, errors may only appear late in the project and be harder to fix.	Use automated or manual testing throughout development.
Crashes	The code may terminate unexpectedly during execution	Intermediate results will be saved at regular intervals to allow the simulation to restart from a recent state rather than from the beginning.

References

[1] A. J. Wood. A totally asymmetric exclusion process with stochastically mediated entrance and exit. *Journal of Physics A: Mathematical and Theoretical*, 42(44):445002, 2009.

[2] Kai Nagel and Michael Schreckenberg. A cellular automaton model for freeway traffic. *Journal de Physique I*, 2(12):2221–2229, 1992.

[3] Helge Holden and Nils Henrik Risebro. Follow-the-leader models can be viewed as a numerical approximation to the lighthill-whitham-richards model for traffic flow, 2017.